SIMATS ENGINEERING

Descriptive Written Exam

Name: M.S.Charishma

Reg.no:192311393

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3

4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3
5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3

10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3
11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3

17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3
18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Code of Conduct:

I __M.S.Charishma___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.S.Charishma

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

ANSWERS

1. Testing as an Engineering Activity

Opening

The banking application should not depend on random checking. Testing has to be planned like an engineering activity, with clear requirements, test objectives and measurable results.

How I would redesign it

- Start from requirements and identify important banking functions such as login, balance enquiry, fund transfer and transaction history.
- Prepare test cases before execution. Include valid, invalid, boundary and negative conditions instead of testing only the normal path.
- Use a defect tracking tool to record the exact steps, expected result, actual result and severity of every failure.
- Follow a common testing standard and review the test cases before execution so that important requirements are not missed.

Example

For a fund transfer of ₹5,000, the tester should check a successful transfer, insufficient balance, wrong account number, network interruption and duplicate submission. These cases are more effective than simply clicking through the application.

Result

This approach gives repeatable testing, better defect detection and clearer evidence of product quality. Ad-hoc testing can therefore be replaced by planned test design, reviews, tracking and controlled execution.

2. Structured Testing Process for an ATM

Requirement stage

First, the withdrawal requirements are studied: card validation, PIN verification, balance checking, cash dispensing and transaction update. Any unclear requirement should be discussed before coding.

During development

Test conditions are prepared for each function. After implementation, unit and integration testing can check whether the ATM, bank server and account database work together correctly.

For withdrawal

- Insert a valid card and enter the correct PIN.

- Try an incorrect PIN and a blocked card.
- Request an amount within the balance and another amount greater than the balance.
- Check minimum and maximum withdrawal limits.
- Verify that the account balance and transaction record are updated correctly.

Why consistency matters

The same process should be followed for every build: plan, design, execute, report defects, retest and perform regression testing. This reduces missed cases and makes results easier to compare. The expected outcome is a more stable ATM with fewer incorrect transactions.

3. Error, Defect and Failure

Meaning

An error is a human mistake made while understanding a requirement or writing code. A defect is the fault introduced into the software because of that mistake. A failure is the incorrect behaviour observed when the defective software is executed.

In the given case

Suppose the requirement says that a user must enter a six-digit PIN, but a developer mistakenly allows five digits. The developer's mistake is an error. The wrong validation in the program is the defect. When a five-digit PIN is accepted during execution, it becomes a failure.

Why the distinction is useful

If testers simply call everything an 'error', developers may not know whether the problem is in the requirement, implementation or observed behaviour. This can delay correction.

Better reporting

A defect report should clearly mention the module, steps to reproduce, expected output, actual output, environment and severity. Such reporting makes communication between testers and developers much clearer.

4. Defect Clustering and Test Prioritization

Main idea

Defect clustering means that a small number of modules often contain a large portion of the defects. Therefore, testing effort should not always be divided equally among all modules.

Applying it

Past defect records can be examined to identify modules with repeated failures. High-risk modules are then given more test cases and earlier execution, while stable low-risk areas receive suitable but smaller coverage.

Example

In an online shopping system, payment and checkout may have many previous defects, whereas the help page may have very few. Payment, checkout and order confirmation should therefore receive stronger regression and negative testing.

Other principles

Testing should also consider that exhaustive testing is impossible and that repeating the same tests may eventually find fewer new defects. New and risk-based test cases should therefore be added.

Benefit

Prioritization helps testers use limited time where failures are most likely and where their effect on users is highest.

5. Defect Classification and Repository

Improving the repository

The company should make the defect repository useful rather than treating it as a storage location. Every defect should be classified according to its origin, type, severity, priority, affected module and release.

Where defects may originate

Typical causes include incorrect requirements, design mistakes, coding errors, interface problems, data issues and environment-related problems.

Using old information

Before preparing tests for a new release, testers can search the repository for earlier defects in the same area. Repeated defects can then become regression test cases.

A simple example

If incorrect tax calculation has appeared in three releases, tax-related boundary values and different salary ranges should be added to the regression suite.

Expected improvement

The repository becomes a source of learning. It helps the team identify weak areas, prevent repeated mistakes and improve future test design.

6. Mobile Banking Fund Transfer Crashes

First investigation

I would first reproduce the crash and collect the conditions under which it occurs. The team should check account balance, transfer amount, network condition, server response, database update and repeated clicks.

Possible causes

- Unexpected or invalid input may not be handled correctly.
- A network timeout may leave the transaction in an incomplete state.
- Two requests may be submitted when the user presses the transfer button repeatedly.
- An exception may occur while updating the transaction record.

Testing the problem

The transfer function should be tested with normal, invalid and boundary amounts, weak network conditions, session expiry and repeated submissions. Logs and defect reports should be used to trace the exact failure.

Prevention

After fixing the defect, the same scenario should be added to regression testing. Automated tests for important transfer rules can also reduce the chance of the problem returning.

Outcome

A systematic investigation finds the cause instead of only retesting the screen. This makes fund transfer safer and more reliable.

7. Testing and Debugging

Testing

Testing is the process of evaluating software to find defects and determine whether it meets its requirements. Testers mainly try to expose problems by executing planned scenarios.

Debugging

Debugging begins after a defect or failure is identified. The developer investigates the code, finds the cause, changes the program and checks whether the correction works.

Consider this situation

If a login page accepts an invalid password, the tester reports the failure with evidence. The developer then traces the authentication logic and corrects the faulty condition. The tester later retests the fix.

Why the confusion causes trouble

If developers start changing code without a clear test result, the real problem may be misunderstood. Similarly, testing alone cannot replace debugging.

Better practice

Testing should discover and document the problem; debugging should locate and correct its cause; retesting and regression testing should then confirm the solution.

8. Cross-Browser Testing

Testing strategy

The e-commerce application should be checked on the browsers and versions that its customers are expected to use. Important functions such as login, product search, cart and payment should be included.

What can go wrong?

Different browsers may interpret CSS, JavaScript, fonts or responsive layouts differently. A feature that works in one browser may therefore appear incorrectly or behave differently in another.

Coverage

- Select the important browser and operating-system combinations.
- Run the same critical test cases across the selected environments.
- Check different screen sizes for responsive pages.
- Repeat important tests after browser-related fixes.

Useful support

Browser developer tools and cross-browser testing platforms can help testers compare behaviour without maintaining every environment manually.

Result

Systematic cross-browser testing reduces environment-specific defects and gives users a more consistent shopping experience.

9. Payroll Calculation Defects

Classification

Incorrect salary calculation caused by wrong logic is mainly a functional or coding defect. Depending on its origin, it may also be traced to a requirement or design defect.

What should be tested

Salary is rarely just one calculation. Basic pay, allowances, deductions, overtime, tax and leave-related rules may all affect the final amount. Each rule needs its own test conditions.

Example

If overtime is calculated only for hours greater than eight but the requirement says overtime starts after nine hours, the program will produce an incorrect salary. Boundary values around the limit can expose this defect.

Prevention

Requirements should be reviewed with developers and testers, calculation rules should have detailed test cases, and important salary scenarios should be retained for regression testing.

Improvement

Using equivalence classes, boundary value analysis and independent review of calculation logic can improve the reliability of future payroll releases.

10. Structured Test Case Design

Why random testing is weak

Random testing may find an obvious problem, but it does not show whether all important requirements have been covered. The same areas may be tested repeatedly while another condition is never checked.

A better method

For each requirement, identify inputs, preconditions, steps and expected results. Give every test case a clear purpose and trace it to the related requirement.

Techniques worth using

- Equivalence partitioning for groups of similar inputs.
- Boundary value analysis for limits and edge conditions.
- Decision table testing when several business rules interact.
- Negative testing for invalid or unexpected inputs.

Example

For a registration form with an age range of 18 to 60, tests should include valid ages, values below 18, values above 60 and the exact boundary values.

Effect

Structured cases provide better coverage, make execution repeatable and make defects easier to reproduce.

11. Healthcare Data Integrity

Analysis

Data inconsistency across healthcare modules is serious because the same patient information may be used by registration, appointment, billing and medical-record functions. The first step is to identify where the values become different.

How defects can arise

The problem may come from incorrect database updates, different validation rules, synchronization failures or one module using an outdated value.

Testing approach

The same patient record should be followed through multiple modules. After an update in one module, testers should verify that the correct value appears wherever that information is supposed to be used.

Example

If a patient's phone number is changed in registration, the updated number should be reflected in the appointment and billing modules when the design requires shared information.

Tracking

Every inconsistency should be reported with the affected patient record or test data, steps, expected value and actual value. This makes the defect easier to reproduce and fix.

12. Preventing Repeated Defects

The problem

Repeated defects usually indicate that the team is fixing individual failures without learning from their history. A defect repository can help break this cycle.

Using previous defects

After each release, testers should review recurring defect types, affected modules and root causes. Important historical defects should be converted into regression tests.

Finding patterns

For example, if several releases contain defects in password validation, the team can examine whether the same requirement, coding practice or review gap is responsible.

What should change

- Tag defects consistently by module, type and cause.
- Review recurring defects during release planning.

- Add high-impact historical defects to regression suites.
- Share common causes with developers and reviewers.

Final effect

The repository then becomes a prevention tool, not merely a list of old bugs. Over time, repeated defects should decrease and the test suite becomes more relevant to the product.

13. Peak-Usage Failure in a Learning System

Where the gap is

Testing only normal daytime usage does not prove that a learning platform can handle peak demand. The system may work correctly for a small class but fail when many students log in, watch videos or submit assignments together.

Test planning

Performance requirements should be identified during planning. Expected concurrent users, response-time targets and important high-load operations should be considered while preparing tests.

A practical test

A load test can simulate many students logging in and accessing course material at the same time. The team can observe response time, server resource usage, error rate and transaction completion.

Why coverage matters

Peak scenarios, sudden increases in users and long-duration usage should be included rather than checking only functional correctness.

Improvement

Performance testing should be repeated after major changes, and the results should guide tuning of the application, database and server configuration. This makes failures less likely when the system is used heavily.

14. Early Tester Involvement in SDLC

Role of the tester

A tester should not enter the project only after coding is complete. Testers can review requirements, identify missing conditions, suggest acceptance criteria and prepare test ideas while development is still in progress.

Why early involvement helps

A requirement mistake found during analysis is usually easier and cheaper to correct than the same mistake discovered after deployment.

Practical example

Suppose a banking requirement says a transfer must be completed quickly but does not specify what should happen when the network disconnects midway. A tester can raise this question early and help the team define the expected behaviour.

Team changes

- Include testers in requirement and design discussions.
- Review acceptance criteria together.
- Share risks and defect trends with developers.
- Plan test data and environments before the build is ready.

Result

Early collaboration reduces misunderstandings, improves testability and supports better quality throughout the SDLC.

15. Incomplete Test Coverage

Finding the missing areas

When a system works for some inputs and fails for others, I would first compare the existing tests with the requirements and possible input conditions. Missing branches, boundaries and negative cases are common coverage gaps.

Improving the test cases

Test cases should cover normal conditions as well as invalid inputs, minimum and maximum values, different user roles and important combinations of conditions.

Example

For a withdrawal limit of ₹20,000, testing only ₹5,000 is insufficient. Values such as ₹0, ₹1, ₹19,999, ₹20,000 and ₹20,001 can reveal different behaviour.

Validation methods

Requirement-to-test traceability, code coverage reports, equivalence partitioning and boundary value analysis can help reveal areas that have not been adequately tested.

Conclusion

Better coverage does not mean testing every possible input. It means selecting representative and high-risk conditions so that important behaviour is actually validated.

16. Improving Defect Reporting

A useful defect report

A defect report should allow another person to reproduce the problem without guessing. It should normally include a short title, environment, preconditions, steps, expected result, actual result, severity and supporting evidence when necessary.

Example

Instead of writing 'login is not working', a better report would state that after entering a valid registered username and password, clicking Login returns the user to the login page instead of opening the dashboard.

Why definitions matter

The team should distinguish a requirement problem, software defect and observed failure. Clear terminology prevents confusion about what needs to be corrected.

Reporting practice

Use a common defect-tracking format and assign priority according to user impact. Developers should update the status after correction, while testers should retest and record the result.

Benefit

Clear reports shorten investigation time, reduce back-and-forth communication and help the team identify recurring problem areas.

17. Process-Based Testing for Consistency

The approach

Testing should follow a common process even when different people work on different modules. The process can define how requirements are reviewed, test cases are prepared, defects are reported and fixes are verified.

Common rules

- Use the same test-case fields and naming style.
- Follow agreed entry and exit criteria.
- Record defects in the same tracking system.
- Perform regression testing after relevant fixes.
- Review important test cases before execution.

Example

If the login, payment and order modules are tested by different testers, each tester should still follow the same reporting and review procedure. Module-specific tests can differ, but the overall quality process remains consistent.

Keeping it consistent

Checklists, peer reviews, templates and periodic test-result reviews help ensure that the process is actually followed.

Outcome

Standardization reduces variation between testers and makes results from different modules easier to understand and compare.

18. Boundary Testing and Edge Cases

Why they are missed

Testers often concentrate on normal values because they represent common usage. However, software frequently fails at limits, unusual combinations and values just outside the permitted range.

Boundary value analysis

This technique focuses on values at and around a limit. If an application accepts marks from 0 to 100, useful tests include 0, 1, 99, 100, -1 and 101.

Application

For an online booking system that allows a maximum of four passengers, the test should not stop at two passengers. Four, five and other invalid values should also be checked.

Making it part of design

During test-case preparation, testers should identify every important minimum, maximum and transition point from the requirements.

Result

Including boundary and edge cases before release improves coverage and helps reveal defects that ordinary positive testing may miss.

19. Defect Documentation and Repository

What is missing

Without proper documentation, the same issue may be investigated repeatedly and useful information can disappear after a release. A structured defect repository solves this by keeping a history of reported problems.

Important information

Each record should contain the defect description, module, steps to reproduce, expected and actual results, severity, priority, status, assigned person and resolution details.

How it supports analysis

Once enough records are collected, the team can identify modules with frequent defects, common causes and defects that remain open for a long time.

Example

If many records are related to payment failures, the team can review that module more closely and strengthen its regression tests before the next release.

Improvement

Regular repository reviews, consistent classification and linking defects to test cases make defect management more useful and improve future testing decisions.

20. Improving Banking Transaction Testing

Redesign

For a banking system with repeated transaction failures, I would begin with the transaction requirements and identify the highest-risk operations. Fund transfer, payment, balance update and transaction history deserve strong coverage because failures can directly affect customers.

Testing in practice

Test cases should include successful transactions, insufficient balance, invalid account details, duplicate requests, network interruption, session expiry and interrupted server responses. Boundary amounts should also be checked.

Example

If a customer presses Transfer twice because the first response is slow, the system should not create two transactions. This scenario should be deliberately tested rather than left to chance.

Supporting activities

Defects should be recorded with complete reproduction details, fixed defects should be retested and related areas should undergo regression testing. Automation can be used for stable, repetitive transaction checks.

Final view

A planned combination of requirement-based, negative, boundary, regression and risk-based testing gives much stronger defect detection than random execution. It also provides better confidence before the banking system is released.